

Kamara Fidele

Mobile Development

African Leadership University

ALU JobLink

A Mobile Platform Connecting ALU Students with Student-Led Startup Internship Opportunities

Technical Report

1. Introduction

Many ALU students struggle to secure internships at established organizations, while several student-led startups within the ALU ecosystem need help in areas such as software development, design, marketing, and content creation but have no structured way to reach students. ALU JobLink is a Flutter mobile application, backed by Firebase, that closes this gap: startups post opportunities, and students discover, filter, and apply for them, with both sides tracking progress in real time.

This report documents the system's architecture, backend structure, state management approach, core workflows, and the design and technical decisions behind them, along with challenges encountered during development and directions for future work.

1.1 Scope

The current build covers the assignment's minimum requirements: authentication and onboarding, startup profiles with a verification flag, opportunity posting, discovery and search, application submission, real-time updates, and Firebase-backed state management. Messaging/chat, push notifications, and an automated admin verification dashboard were treated as stretch goals and are discussed in the Future Improvements section rather than implemented in this version.

2. System Architecture

ALU JobLink follows a three-tier architecture that keeps the UI, application state, and backend cleanly separated, which made the codebase easier to reason about and to extend one screen at a time during development.

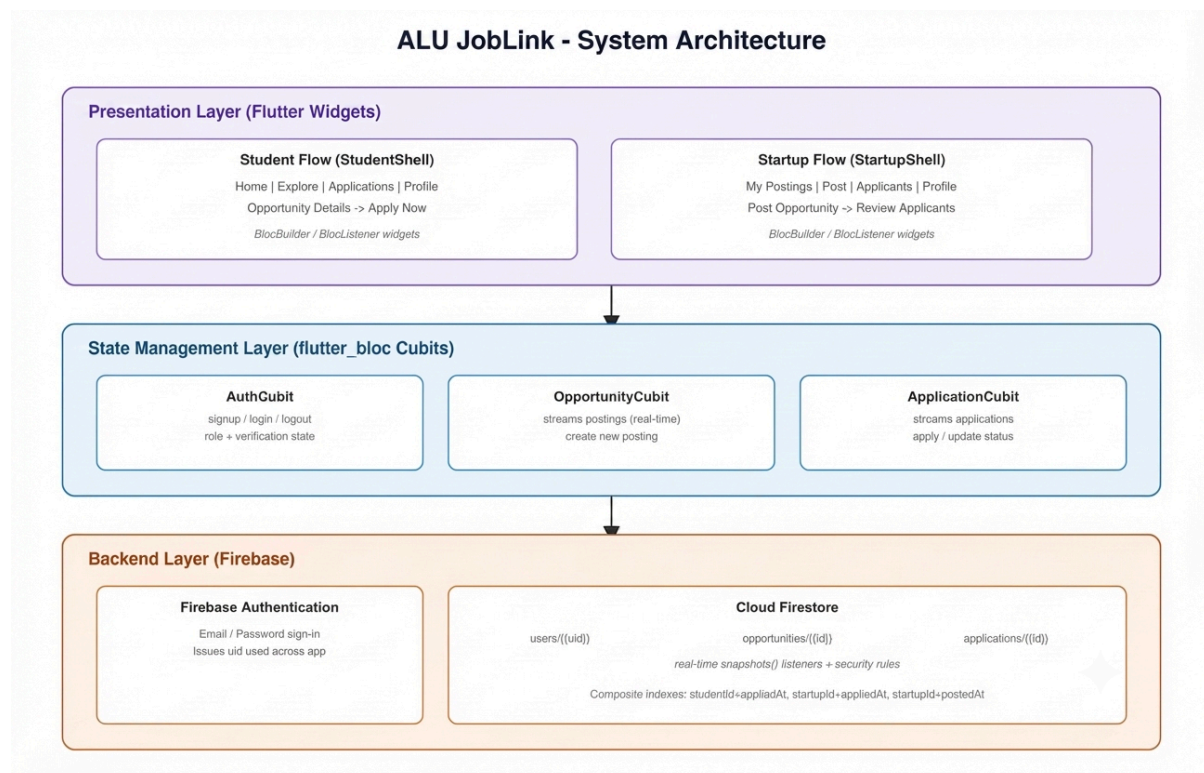


Figure 1. Three-tier architecture of ALU JobLink.

2.1 Presentation Layer

Screens are grouped into a StudentShell and a StartupShell, each holding a bottom navigation bar over four tabs (mirroring the assignment's sample UI). Widgets in this layer are intentionally 'dumb': they read state from a Cubit through BlocBuilder and forward user actions (button taps, form submissions) back to Cubit methods. No Firebase calls happen directly inside a widget's build method.

2.2 State Management Layer

Three Cubits (flutter_bloc) own all business logic: AuthCubit, OpportunityCubit, and ApplicationCubit. Cubit was chosen over full BLoC because the app's state transitions are simple request/response or stream-driven updates with no need for an event-mapping layer; Cubit keeps the code that triggers a state change next to the method that was called, which is quicker to trace and to explain.

OpportunityCubit and ApplicationCubit are instantiated once per shell (in StudentShell/StartupShell) rather than once per screen or globally at the app root. This matters for two reasons: first, a student and a startup need differently scoped data (all opportunities vs. only opportunities the logged-in startup posted); second, because the shells use an IndexedStack to keep all four tabs alive in memory, switching tabs does not tear down and recreate the Cubit, so a single Firestore listener stays open per shell rather than one per screen visit.

2.3 Backend Layer

Firebase Authentication handles identity and issues each user a uid, which is used throughout Firestore as a foreign key. Cloud Firestore stores all application data across three collections described in Section 3, and Firestore security rules (Appendix A) enforce that a user can only read or modify data they are entitled to.

3. Firebase Backend Structure and Database Schema

Firestore is a NoSQL document database: it has no joins, so the schema was designed around the queries the app actually needs to run, rather than around normalized relational tables.

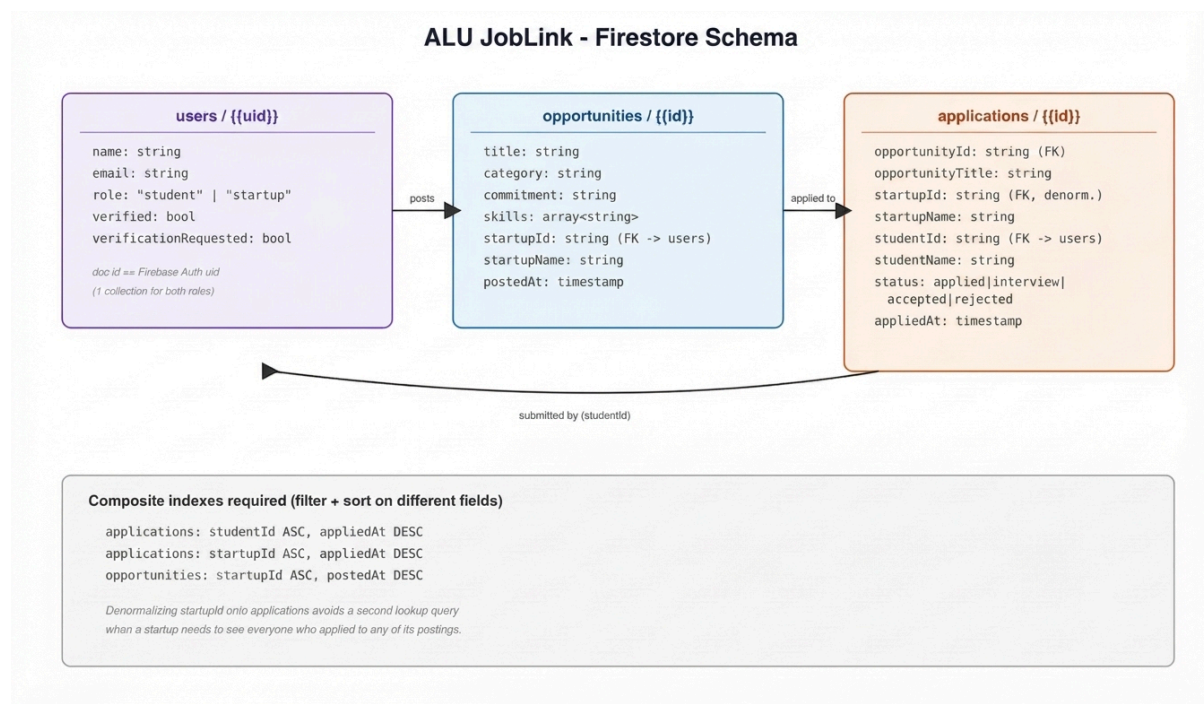


Figure 2. Firestore collections and their relationships.

3.1 users/{uid}

A single collection holds both students and startups, distinguished by a role field. This was a deliberate simplification: the two roles share almost all profile fields (name, email), and splitting them into separate collections would have duplicated the authentication-linking logic for no real benefit at this scale. A verified boolean (true by default for students, false by default for startups) supports the requirement that only recognized startups can post opportunities.

3.2 opportunities/{id}

Each posting stores the startupId and startupName directly, rather than only a reference to the startup's user document. Copying the startup's name onto every posting (denormalization) means the opportunity list can render startup names without a second read per card - a standard NoSQL trade-off of small write-time duplication for much simpler, cheaper reads.

3.3 applications/{id}

Applications denormalize startupId as well, even though it could technically be looked up through the referenced opportunity. This lets the startup side query 'everyone who applied to any of my postings' with a single where(startupId == uid) clause instead of first fetching all of a startup's opportunity IDs and then querying applications for each one.

3.4 Composite Indexes

Three queries filter on one field while ordering by another (e.g., where studentId == uid, order by appliedAt descending). Firestore cannot satisfy this combination with its default single-field indexes, so composite indexes were defined in firestore.indexes.json for: (a) applications by studentId + appliedAt, (b) applications by startupId + appliedAt, and (c) opportunities by startupId + postedAt. Firestore also raises a runtime error containing a ready-made console link the first time an un-indexed query executes, which is how the missing indexes were originally discovered during testing.

4. State Management Approach

The app uses flutter_bloc's Cubit class throughout. Each Cubit exposes a single immutable state object and one method per user-triggered action; UI widgets never mutate state directly.

4.1 AuthCubit

AuthState carries a status enum (unauthenticated, loading, authenticatedStudent, authenticatedStartup), the user's uid and name, their verified flag, and an optional errorMessage. A typical transition sequence during login is unauthenticated -> loading -> authenticatedStudent (or back to unauthenticated with an errorMessage set, on failure). AuthGate, a small widget at the app root, watches this state with BlocBuilder and swaps between the Login screen, StudentShell, and StartupShell accordingly - routing is a pure function of state, not of imperative navigation calls scattered across the app.

4.2 OpportunityCubit and ApplicationCubit

Both Cubits subscribe to a Firestore .snapshots() stream (not a one-time .get()) inside their load methods, and emit a new state every time the stream fires. This is what satisfies the assignment's real-time/dynamic-updates requirement: when a startup posts a new opportunity, every student currently on the Home or Explore tab sees it appear without pulling to refresh, because the same underlying stream is already open and pushing changes into the Cubit.

4.3 BlocBuilder vs. BlocListener

BlocBuilder is used wherever a state change should redraw part of the UI (an opportunity list, an application's status chip). BlocListener is used for one-off side effects that should not trigger a rebuild, such as showing a Snackbar when AuthCubit emits an error message during login or signup - keeping these responsibilities separate avoids duplicate SnackBars firing on every unrelated rebuild.

5. Application Workflows

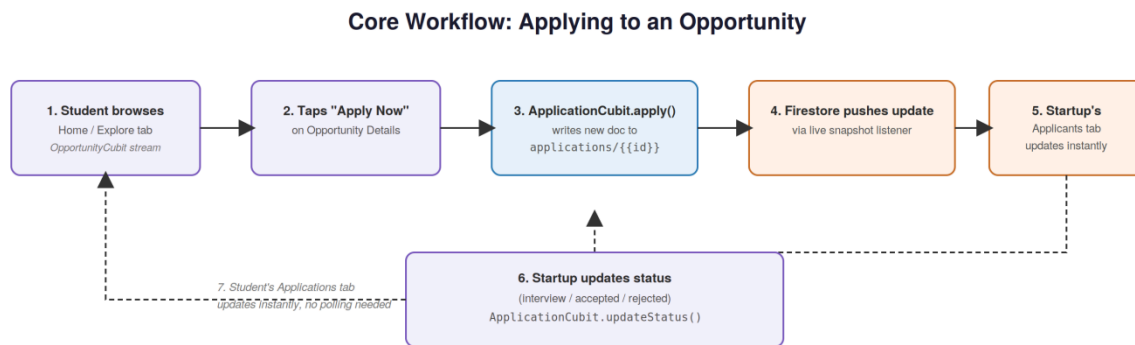


Figure 3. End-to-end workflow when a student applies to an opportunity.

5.1 Student Workflow

- Sign up as a student (auto-verified) or log in.
- Browse recent postings on Home, or filter by category on Explore.
- Open Opportunity Details to read the full description and required skills.
- Tap Apply Now, which writes a new document to the applications collection.
- Track status changes (applied -> interview -> accepted/rejected) on the Applications tab in real time.

5.2 Startup Workflow

- Sign up as a startup (unverified by default) or log in.
- Request verification from the Profile tab; an admin currently confirms this manually in the Firebase console (see Section 9).
- Post an opportunity with title, category, commitment, description, and required skills.
- Review applicants on the Applicants tab as they arrive.
- Update an applicant's status, which the student sees update on their end instantly.

6. UI/UX Reasoning

The four-tab bottom navigation pattern (Home, Explore, Applications, Profile for students; My Postings, Post, Applicants, Profile for startups) was chosen because it mirrors navigation students already know from apps like Instagram and LinkedIn, minimizing the learning curve for a campus audience that will use this occasionally rather than daily.

Unverified startups still see the Post Opportunity form, but with a visible banner explaining that their posting will not go live until verification completes. This was a deliberate choice over hiding the form entirely: it lets a founder prepare their first posting immediately after

signing up, rather than being blocked with no visible next step, while still being honest about the current limitation.

7. Scalability Considerations

- Firestore scales horizontally by design, so growth in the number of students, startups, or postings does not require managing servers directly.
- Composite indexes keep query cost roughly constant as collections grow, avoiding full-collection scans on every Home/Explore/Applicants load.
- Denormalizing startupId onto applications avoids one query per opportunity that would otherwise grow linearly with a startup's number of postings.

8. Challenges Encountered and Lessons Learned

8.1 Misleading authentication error

During testing, login attempts on an Android emulator failed with 'The supplied auth credential is incorrect, malformed or has expired,' even though the email and password were correct and the account existed. The Android log traced the failure to a reCAPTCHA/Play Integrity check (RecaptchaCallWrapper) rather than to the credentials themselves. The root cause was that the Android app's debug signing certificate (SHA-1/SHA-256 fingerprints) had not yet been registered with the Firebase project. Running ./gradlew signingReport and adding the resulting fingerprints in Firebase Console resolved it.

8.2 Lesson Learned

Modern mobile auth providers wrap device-attestation layers (Play Integrity) around what looks like a plain username/password flow, and the resulting error messages describe the symptom, not the root cause - reading the surrounding platform logs, not just the exception message, was necessary to diagnose this correctly.

9. Limitations

- No automated unit or widget tests in the current version.
- Startup verification is manual (an admin toggles a Firestore field) rather than backed by an admin dashboard.
- No push notifications when an application's status changes; students must open the app to see updates.

10. Future Improvements

- Build a simple admin dashboard for startup verification instead of manual Firestore edits.

- Add push notifications (Firebase Cloud Messaging) for application status changes.